# Exercise 5 – Evaluation I

**Introduction to Machine Learning**

*Hint: useful* `Python` *libraries*

```
# Consider the following libraries for this exercise sheet:

# general
import numpy as np
import pandas as pd
import math

# plots
import matplotlib.pyplot as plt

# sklearn
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
```

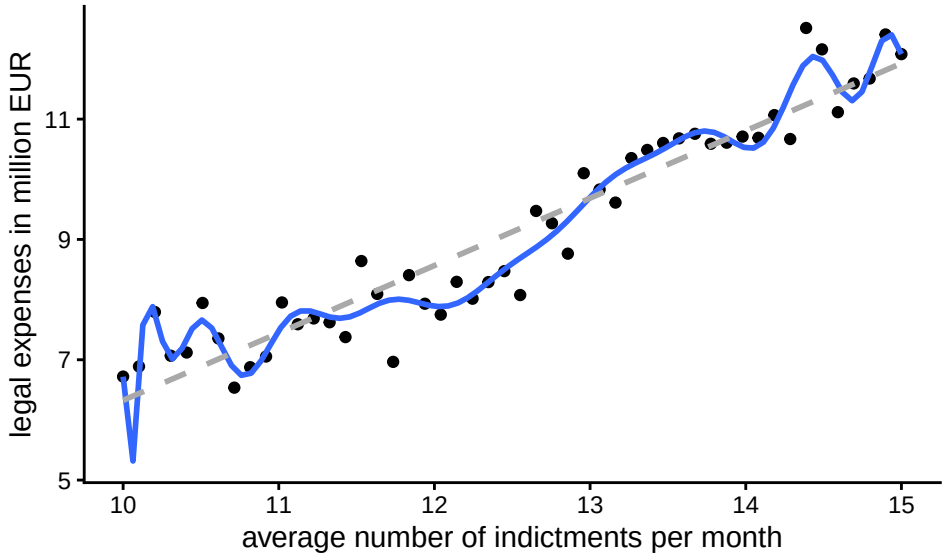
## Exercise 1: Evaluating regression learners

> Learning goals
>
> 1. Understand importance of using test error for evaluation
> 2. Discuss choice of performance metric in given situation

Imagine you work for a data science start-up and sell turn-key statistical models. Based on a set of training data, you develop a regression model to predict a customer's legal expenses from the average monthly number of indictments brought against their firm.

a) Due to the financial sensitivity of the situation, you opt for a very flexible learner that fits the customer's data ($n_{train} = 50$ observations) well, and end up with a degree-21 polynomial (blue, solid). Your colleague is skeptical and argues for a much simpler linear

learner (gray, dashed). Which of the models will have a lower empirical risk if standard $L2$ loss is used?




b) Why might evaluation based on training error not be a good idea here?

c) Evaluate both learners using

(i) mean squared error (MSE), and

(ii) mean absolute error (MAE).

First compute both measures **by hand** for the following small test set of five customers, where $y^{(i)}$ is the true legal expense and $\hat{y}^{(i)}_{\text{poly}}$, $\hat{y}^{(i)}_{\text{lin}}$ are the predictions of the polynomial and the linear learner:

| Customer $i$ | $y^{(i)}$ | $\hat{y}^{(i)}_{\text{poly}}$ | $\hat{y}^{(i)}_{\text{lin}}$ |
|---|---|---|---|
| 1 | 10 | 11 | 10 |
| 2 | 12 | 13 | 12 |
| 3 | 14 | 15 | 14 |
| 4 | 16 | 17 | 16 |
| 5 | 20 | 19 | 17 |

State your performance assessment and explain potential differences. You can then verify the behaviour on the full test set using the code below.

**R**

```r
set.seed(123)
x_train <- seq(10, 15, length.out = 50)
y_train <- 10 + 3 * sin(0.15 * pi * x_train) + rnorm(length(x_train), sd = 0.5)
data_train <- data.frame(x = x_train, y = y_train)
set.seed(321)
x_test <- seq(10, 15, length.out = 10)
y_test <- 10 + 3 * sin(0.15 * pi * x_test) + rnorm(length(x_test), sd = 0.5)
data_test <- data.frame(x = x_test, y = y_test)
```

#### Python

```python
np.random.seed(43)
x_train = np.linspace(10, 15, num=50)
y_train = 10 + 3 * np.sin(0.15 * math.pi * x_train)
y_train += np.random.normal(loc=0.0, scale=0.5, size=len(x_train))
data_train = pd.DataFrame({"y": y_train, "x": x_train})
np.random.seed(2238)
x_test = np.linspace(10, 15, num=50)
y_test = 10 + 3 * np.sin(0.15 * math.pi * x_test)
y_test += np.random.normal(loc=0.0, scale=0.5, size=len(x_test))
data_test = pd.DataFrame({"y": y_test, "x": x_test})
```

### Exercise 2: Importance of train-test split

> Learning goals
>
> 1) Understand how strongly performance estimates can depend on data samples
> 2) Explain bias-variance trade-off of GE estimator for repeated sampling across different train/test splits

We consider the `CaliforniaHousing` data for which we would like to predict the median house value (`MedHouseVal`) from the median income in the neighborhood (`MedInc`).

#### R

Get data

```r
# Adapt to version available on sklearn following description from
# https://gist.github.com/bgreenwell/b1330460eec5acf1c81fae71902e331c

setwd(tempdir())
# (figshare mirror of cal_housing.tgz -- the same archive sklearn downloads;
```

```r
# the original https://www.dcc.fc.up.pt/~ltorgo/Regression/cal_housing.tgz is offline)
url <- "https://ndownloader.figshare.com/files/5976036"
download.file(url, destfile = "cal.tar.gz")
untar("cal.tar.gz")

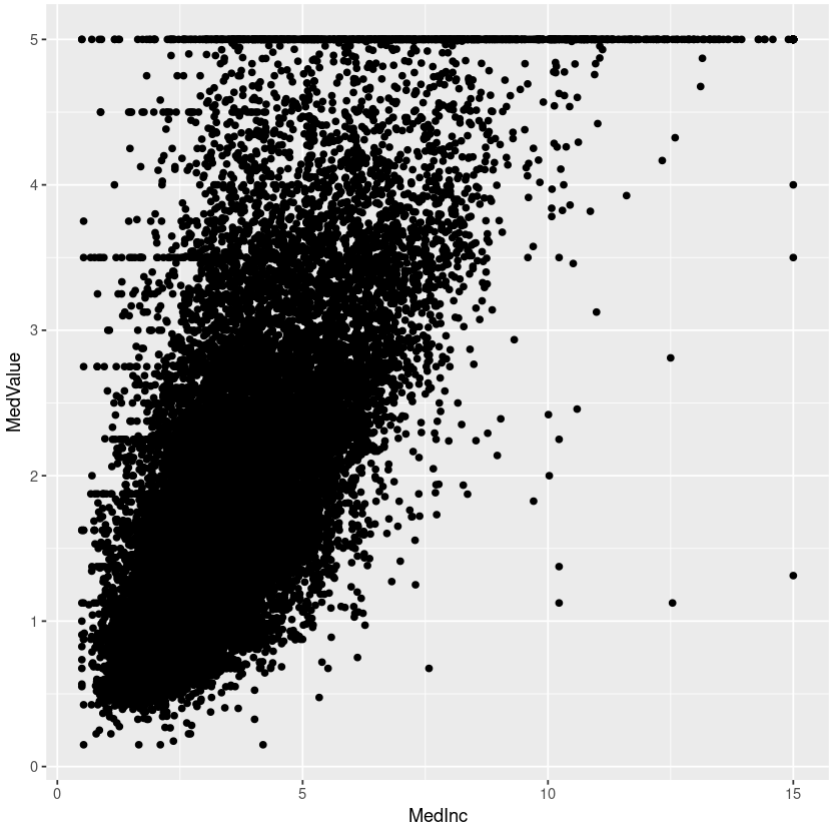
# Read the data into R and provide column names
df_california <- read.csv("CaliforniaHousing//cal_housing.data", header = FALSE)
columns_index <- c(9, 8, 3, 4, 5, 6, 7, 2, 1)
df_california <- df_california[, columns_index]
names(df_california) <- c(
    "MedValue",
    "MedInc",
    "HouseAge",
    "AveRooms",
    "AveBedrms",
    "Population",
    "AveOccup",
    "Latitude",
    "Longitude"
)
df_california$MedValue <- df_california$MedValue / 100000
df_california <- df_california[, c("MedInc", "MedValue")]
head(df_california)
```

A data.frame: 6 × 2

| | MedInc <dbl> | MedValue <dbl> |
|---|---|---|
| 1 | 8.3252 | 4.526 |
| 2 | 8.3014 | 3.585 |
| 3 | 7.2574 | 3.521 |
| 4 | 5.6431 | 3.413 |
| 5 | 3.8462 | 3.422 |
| 6 | 4.0368 | 2.697 |

Inspect

```r
ggplot(df_california, aes(x = MedInc, y = MedValue)) +
    geom_point()
```




### Python

Get data

```python
from sklearn.datasets import fetch_california_housing

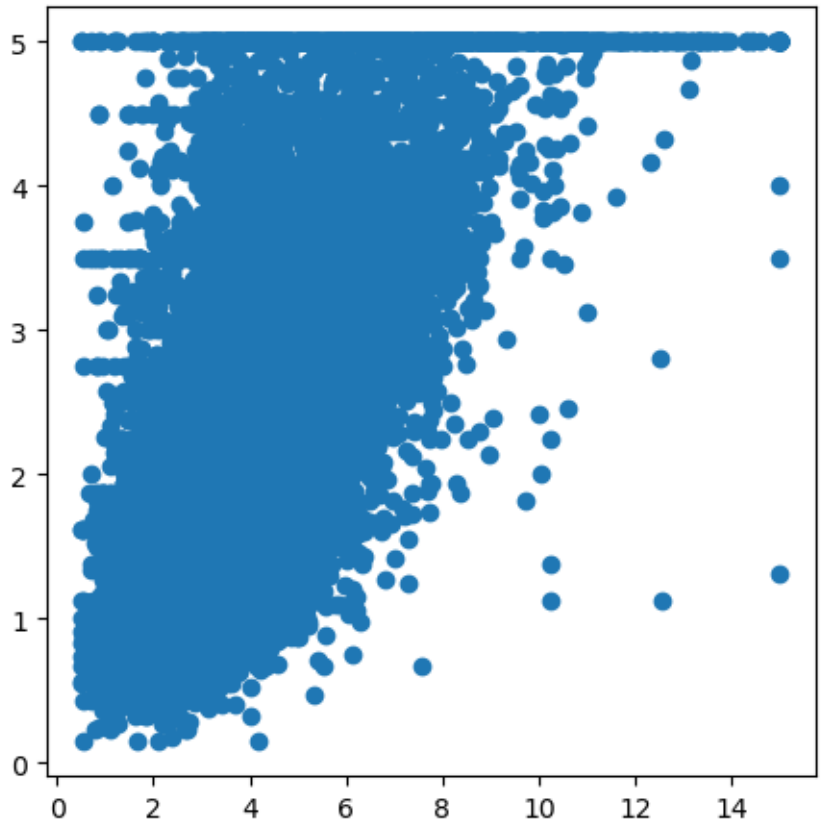
dataset_california = fetch_california_housing(as_frame=True)
df_california = dataset_california.frame.loc[:, ["MedInc", "MedHouseVal"]]
df_california.head()
```

| | MedInc | MedHouseVal |
|---|---|---|
| 0 | 8.3252 | 4.526 |
| 1 | 8.3014 | 3.585 |
| 2 | 7.2574 | 3.521 |

| | MedInc | MedHouseVal |
|---|---|---|
| 3 | 5.6431 | 3.413 |
| 4 | 3.8462 | 3.422 |

Inspect

```
# instantiate plot
fig = plt.figure(figsize=(4, 4))
# Creating axes instance
ax = fig.add_axes([0, 0, 1, 1])
# Creating plot
sp = ax.scatter(
    x=df_california.loc[:, ["MedInc"]], y=df_california.loc[:, ["MedHouseVal"]]
)
plt.show()
```




a) Use the first 100 observations as training data to compute a linear model and evaluate the performance of your learner on the remaining data using MSE.

b) What might be disadvantageous about the previous train-test split?

c) Now, sample your training observations from the data set at random. Use a share of 0.1 through 0.9, in 0.1 steps, of observations for training and repeat this procedure ten times. Afterwards, plot the resulting test errors (in terms of MSE) in a suitable manner.

*Hint*

**R**

`rsmp` is a convenient function for splitting data – you will want to choose the "holdout" strategy. Afterwards, `resample` can be used to repeatedly fit the learner.

**Python**

`from sklearn.model_selection import train_test_split` is a convenient function for splitting data. It has an optional parameter `random_state`, which can be used to split the data randomly in each iteration.

d) Interpret the findings from the previous question.

## Exercise 3: Choosing the right performance measure

The whole exercise is only for lecture group B!

Learning goals

Choose a suitable regression performance measure for a given situation

You work at an online shop and have trained a regression model that predicts the **weekly demand** (number of units sold) of each product in the catalog. Depending on what the predictions are used for, a different performance measure is appropriate. For each of the following use cases of the *same* model, decide which measure from the lecture – **MSE**, **RMSE**, **MAE**, **MAPE**, **Spearman's** $\rho$, or $R^2$ – you would report, and justify your choice in one or two sentences.

a) To decide which products to feature on the homepage, you only need the products **ranked** correctly by predicted demand – the exact predicted numbers do not matter.

b) The catalog contains both niche products (a few units per week) and blockbusters (thousands per week). An absolute error of 10 units is negligible for a blockbuster but severe for a niche product, so you care about the **relative** error.

c) You want to explain to your (non-technical) manager how much better the model is than simply always predicting each product's **average past demand**.

## Exercise 4: Properties of $R^2$

The whole exercise is only for lecture group A!

Learning goals

1. Interpret $R^2$ as a comparison to a featureless baseline model
2. Derive the range of $R^2$ on training data
3. Understand why $R^2$ can become negative on test data

Recall the coefficient of determination

$$R^2 = 1 - \frac{\mathrm{SSE}_{\mathrm{model}}}{\mathrm{SSE}_{\mathrm{const}}}, \qquad \mathrm{SSE}_{\mathrm{model}} = \sum_{i=1}^{n} \left(y^{(i)} - \hat{y}^{(i)}\right)^2, \qquad \mathrm{SSE}_{\mathrm{const}} = \sum_{i=1}^{n} \left(y^{(i)} - \bar{y}\right)^2,$$

where $\hat{y}^{(i)}$ are the model's predictions and $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y^{(i)}$.

a) The denominator $\mathrm{SSE}_{\mathrm{const}}$ is the SSE of a specific *featureless* baseline model. Which model is it, what does it predict, and in which sense does $R^2$ measure a model's performance *relative to* this baseline?

b) Show that for a linear model with intercept, fitted by least squares **on the same data** on which $R^2$ is computed, we always have $R^2 \in [0, 1]$.

*Hint*

The baseline model from a) is itself a member of the linear-model hypothesis space. What does least-squares fitting then imply for $\mathrm{SSE}_{\mathrm{model}}$ compared to $\mathrm{SSE}_{\mathrm{const}}$?

c) Now the model is fitted on training data, but $R^2$ is computed on separate **test** data. Show that $R^2$ can become negative, state the condition for this, and interpret what a negative $R^2$ means.